



نظري

2

8

960 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Amdahl law & Group messages

د. روان قرعوني



RB Informatcs; 21/05/2025

محتوى مجاني غير مخصص للبيع التجاري

برمجة تفرعية

قانون Amdahl للاستجابة العظمى

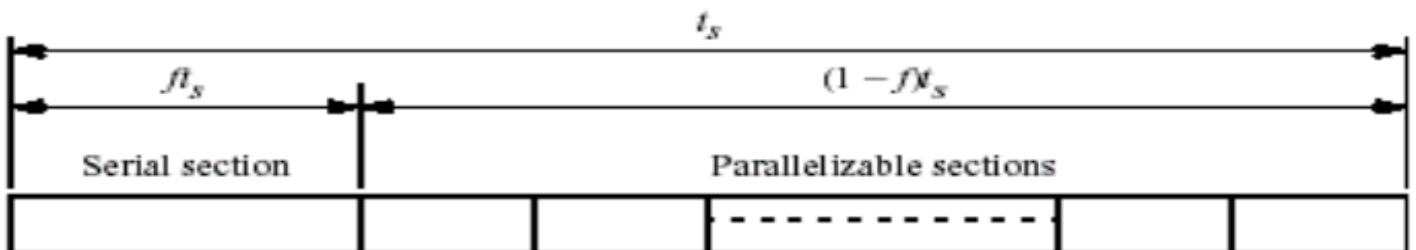
حسب قانون Amdahl فإنه يوجد دائماً في كل برنامج جزء تسلسلي يجب تنفيذه على التسلسل، بما أنه لا يمكن للبرامج دائماً أن تتفرع بشكل كامل. فيجب أن يكون هناك اعتماد على جزء تسلسلي. إن الزمن الكلي لمسألة هو t_s



الزمن التفرعي يقول لذي جزء تسلسلي f + جزء تفرعي $(f-1)$

وبحيث ان الجزء التسلسلي يجب استهلاكه كاملاً

اما الجزء التفرعي يمكن تجزيته من خلال جعله يعمل على عدة معالجات



إذا كان لذي برنامج معين ونريد تمثيل الزمن اللازم لتنفيذه نقوم إما باستخدام البرمجة التسلسلية t_s او باستخدام

$$t_s = f * t_s + (1 - f) * t_s$$

حيث f هي نسبة التسلسل بالبرنامج فنجد أن

$f*t_s$ هي زمن تنفيذ الجزء التسلسلي

$t_s(1-f)$ هي زمن تنفيذ الجزء التفرعي

■ عندها نستطيع توزيع الجزء التفرعي على عدة معالجات ويمكننا حساب زمن تنفيذه من خلال

$$\frac{(1-f)ts}{p} \quad \text{حيث } p: \text{ عدد المعالجات}$$

مثال : لو كان لدي برنامج نسبة التسلسل فيه 10% : إذا ستكون نسبة الجزء التفرعي منه : 90% = (1-10)

$$ts = f * ts + (1 - f) * ts$$

$$tp = f * ts + \frac{(1-f)ts}{p} \quad \text{الزمن التفرعي الكلي :}$$

ومنه تكون الاستجابة العظمى:

$$S(p) = \frac{ts}{f * ts + \frac{(1-f) * ts}{p}}$$

بعد القيام بتوحيد المقامات يصبح

$$\frac{p * ts}{p * f * ts + (1 - f) * ts}$$

نقوم بإخراج الـ t_s ك عامل مشترك واختصارها من البسط والمقام يصبح

$$\frac{p}{p * f + (1 - f)} =$$

الآن نقوم بإخراج الـ p كعامل مشترك

$$= \frac{1}{f + \frac{1}{p} - \frac{f}{p}}$$

ولدينا القانون الرياضي:

$$\lim_{p \rightarrow \infty} \frac{1}{p} = 0$$

بتطبيق القاعدة على قانون الاستجابة (قانون Amdahl)

$$\lim_{p \rightarrow \infty} s(p) = \frac{1}{f}$$

$p \rightarrow \infty$ تعني أن عدد المعالجات لا نهائي

■ نجد أنه عندما يكون عدد المعالجات يسعى إلى قيمة كبيرة تكون الاستجابة مساوية لمقلوب الجزء التسلسلي من البرنامج ومنه نلاحظ أن نسبة التسلسل بالبرنامج هو العامل الأساسي في حساب الاستجابة العظمى



■ مثلاً: عندما قلنا أن f هي 10% عند الاستجابة العظمى

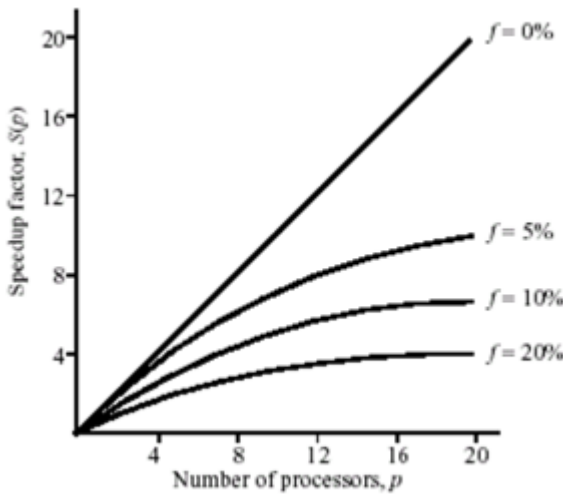
$$s(p) = \frac{1}{\frac{10}{100}} = 10$$

■ ومنه مهما زادت عدد المعالجات عندي لن أصل لزمن استجابة أعلى وأفضل من 10

■ عندما تكون كامل المسألة متفرعة (كلما زادت عدد المعالجات كلما حصلنا على استجابة أعلى)

■ إذا كانت نسبة التسلسل 0% سيصبح زمن الاستجابة لا نهائي (هام)

■ نلاحظ العلاقة بين زيادة عدد المعالجات والاستجابة في المخطط التالي على اليسار:



■ حسب قانون Amdahl كلما زاد عدد المعالجات p

زادت الاستجابة $s(p)$ ولكن هذا محكوم بطبيعة المسألة

■ نلاحظ عند ال 0% وصلنا لحد لا يمكن ان تزيد الاستجابة

مهما زدنا عدد المعالجات أي أنه عندما $f = 0$

(لا يوجد تسلسل بالبرنامج) ويسعى الخط للنهاية

■ هل يعني هذا أنه في حالة عدم وجود جزء تسلسلي،

كلما ازداد عدد المعالجات زادت السرعة بدون وجود حد

لزيادة عدد المعالجات؟!

الجواب: لا، تكون زيادة عدد المعالجات متعلقة أيضاً

بطبيعة المسألة.

مثال للتوضيح: لدينا مصفوفة مربعة 4 * 4

ونريد زيادة كل قيمة في هذه المصفوفة بمقدار 1 لا يمكن استخدام أكثر من 16 معالج (لكل خانة معالج) وإلا تكون

الزيادة غير مفيدة بينما في حال لدينا مصفوفة كبيرة جداً (مليار * مليار)

هنا يمكننا زيادة عدد المعالجات أكثر للوصول لاستجابة عظمية .

انشاء المهام

1. Static process creation

تستخدم عادة (MPI) أنشئ المهام من بداية البرنامج ، كإنشاء 10 معالجات أحتاجها لحظة إقلاع البرنامج

2. Dynamic process Creation

تستخدم عادة (PVM) حسب الحاجة يتم إنشاء المهمة . أي أن المهمة الأب تعمل ، ثم تقوم ب Spawn وقد تحتاج

لإدخال من المستخدم، وعلى أساسها تعرف كم ابن ستحتاج

توابع الارسال والاستقبال

Message Tag-

-توابع تجميعية

-توابع بسيطة

broadcast, multicast, مثل

1-متزامنة

Scatter,gather,Reduce

2-غير متزامنة

مثل: send , recv

التوابع البسيطة

1. في حال التنفيذ المتزامن

تقوم كل تعليمة بانتظار مقابلتها حتى يتم استدعائها

توضيح:

بداية يجب معرفة ان كل عملية تواصل بين المهام تتم باستخدام كلا التابعين

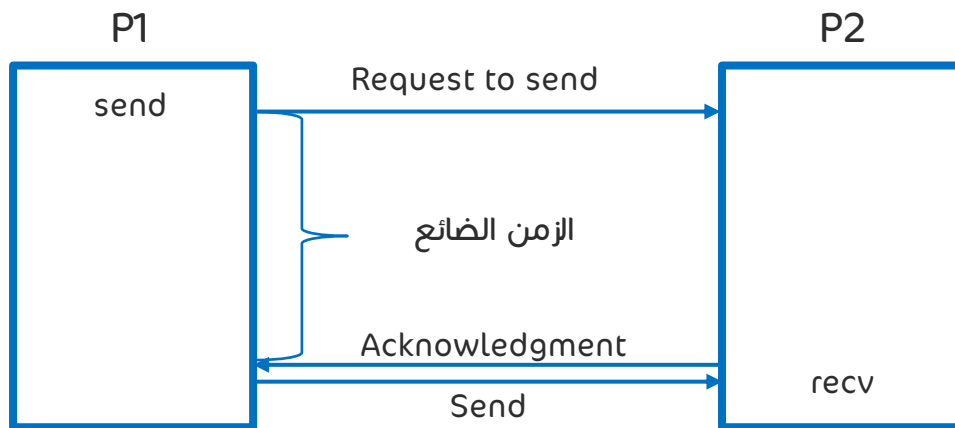
Send

recv

حيث نقوم باستدعاء كل تابع في مهمة

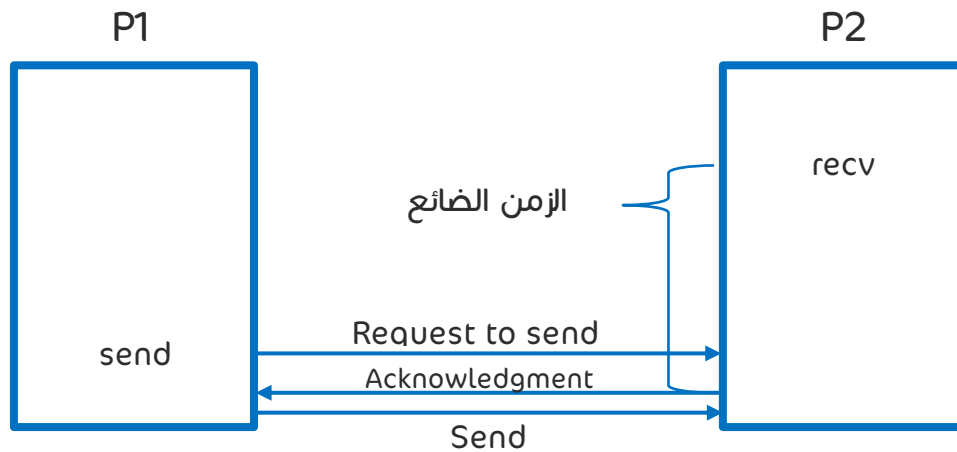
مثال 1:

قامت العملية الأولى بتنفيذ التابع Send بإرسال "request to send" للعملية الثانية ويتوقف التنفيذ في العملية الأولى حتى تقوم العملية الثانية بإرسال acknowledgment من خلال استدعاء تابع الـ recv ثم تقوم التعليمة الأولى بعملية الإرسال send وتستأنف التنفيذ



- بعبارة أبسط تقوم المهمة الأولى بطلب الموافقة على إرسال بيانات معينة، وتقوم بإيقاف التنفيذ في انتظار الموافقة من التعليمة الثانية على قبول هذه البيانات ثم يستأنف كليهما التنفيذ لكن نلاحظ وجود زمن ضائع بين إرسال العملية الأولى لطلب الإرسال وقبول التعليمة الثانية له.

- ماذا يحدث في حال قمنا باستدعاء التابع recv في المهمة الثانية قبل استدعاء تابع send في المهمة الأولى؟
- سيتوقف التنفيذ في العملية الثانية حتى وصول طلب الإرسال من العملية الأولى، بعدها تقوم العملية الأولى بإرسال البيانات، ويستأنف كل من العمليتين التنفيذ وبسبب ذلك نلاحظ وجود زمن ضائع وهذه هي مشكلة التتابع المتزامنة ضياع الوقت

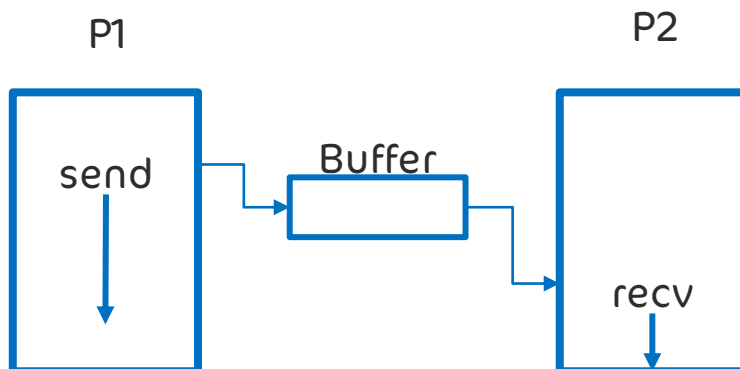


- **ملاحظة هامة:** يجب مقابلة كل تابع recv بتابع send وذلك لأنه في حالة وجود أحدهما سيتوقف التنفيذ في العملية الحالية حتى يتم تنفيذ التابع المقابل في عملية أخرى وفي حال عدم وجوده فإن التنفيذ لن يستأنف

2. في حال التنفيذ غير المتزامن

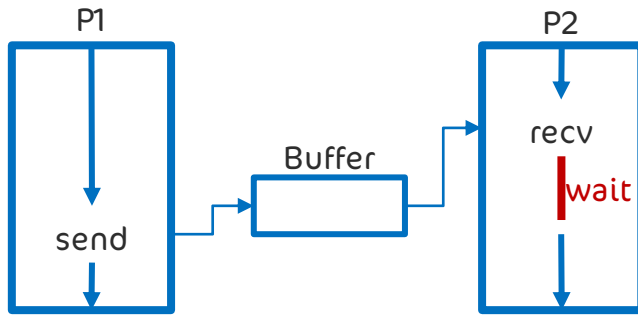
نخزن المطلوب بـ Buffer والمرسل أو المستقبل يقرأ منها مباشرة

مشكلتها: من الممكن ان تمتلئ ال Buffer اذ عند الامتلاء يكون الحل بالعودة لطريقة الانتظار



حالة ال Send قبل ال recv

يقوم ال send بإرسال رسالة الى ال P2 وعند عدم تمكن ال P2 من الاستقبال يقوم بتخزين الرسالة بال Buffer وعندها يقرأ ال recv من ال Buffer مباشرة

حالة ال Recv قبل ال Send

- المشكلة بوجود تعليمات بالأسطر التي سيقوم ال recv بتنفيذها كونه لن يتوقف عن العمل والبيانات المستخدمة في هذه الأسطر تعتمد على النتيجة التي يجب أخذها من تعليمة ال send في ال P1
- إذاً ما هو الحل؟ الحل بعمل Wait سنتكلم عنها لاحقاً
- ويتم وضعها قبل أسطر المعلومة التي سأحتاجها من ال Send
- من الممكن أيضاً وضع ال recv لاحقاً في حال عدم اعتماد الأسطر على البيانات
- المهم وضع ال recv قبل لحظة استخدام القيمة المرادة في ال P1
- من الممكن وضع ال recv قبل ال wait مباشرة

Message Tag

لكل رسالة Tag يفيد في تمييز الرسائل عن بعضها, لكل رسالة أو مجموعة رسائل tag خاصة بهم , ومنه ال recv ستعلم من خلال ال tag أي رسالة ستقرأ (من خلال أحد ال parameter في تابع ال recv)

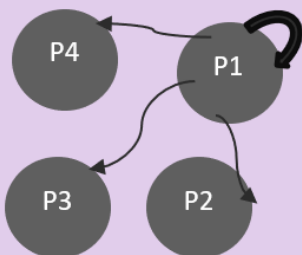
التوابع التجميعية

أحياناً أرغب بإرسال نفس الرسالة لأكثر من مهمة فبالحالة الطبيعية قد أقوم بإنشاء حلقة for بها تعليمة ال Send لكل مهمة.

لذلك كان الحل بالتوابع التجميعية

1-Broadcast

عندي مجموعة من المهام ضمن غروب محدد وأريد إرسال رسالة من P1 لكل المهام الباقيين فهو تابع إرسال معلومة لعدة مهام بنفس الوقت ومن ضمنهم نفسه فائدتها: توفير وقت لان زمن الارسال قليل



2-Multicast

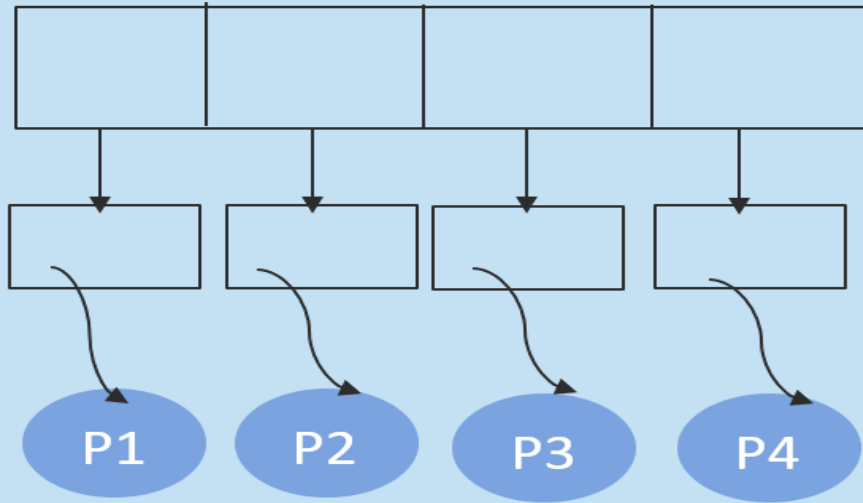
نفس آلية ال Broadcast لكن هنا يمكن تخصيص المهمة التي أريدها ان تستقبل الرسالة ك (P1,P2) من ضمن ال Parameter الخاصة فيها هناك ID المهمة نحصل عليه من ال response spoon

3-Scatter

مثلا عندي مصفوفة و4 مهام

يوفر وقت تقسيم المصفوفة وارسالها جزء جزء يمكن التحكم بعملية التقسيم مثلا كل قسمين من المصفوفة بقسم واحد

يقوم بتهيئة كل معلومة في المصفوفة لوحدها ويرسل كل معلومة من المصفوفة لوحدها ويرسل كل معلومة لنفسه وللمهام الأخرى



مثل مصفوفة ب 100 عنصر ولدي 4 مهام لوكان التقسيم بالتساوي عندها نعطي 25 عنصر لكل مهمة

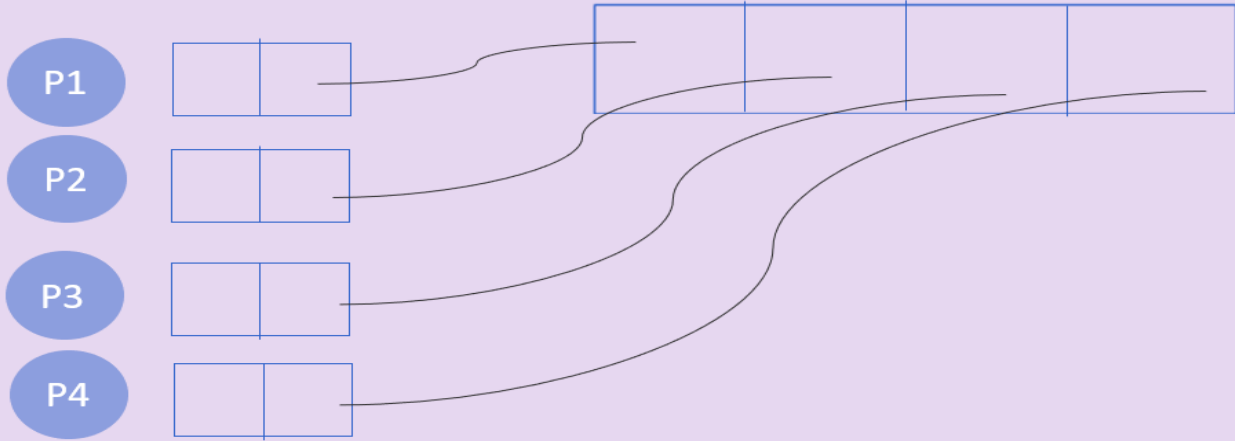
يتم القيام بال Scatter في المرسل والمستقبل بدل من Send, recv تقوم بالتوزيع Scatter تستقبل الاجزاء

يعلم الله إصرارك، ويعلم محاولتك العديدة للوصول، ويعلم الحلم الذي تسعى جاهداً لتناله، ستصل إليه بعون من الله واجتهاد منك.



4-gather

يكون عندي مجموعة مهام ، لكل مهمة قيمة أو اثنتين ، فيقوم هو بتجميع الأجزاء في مصفوفة واحدة مع الحفاظ على ترتيب المهمات .
التعليمة نفسها gather في المرسل والمستقبل

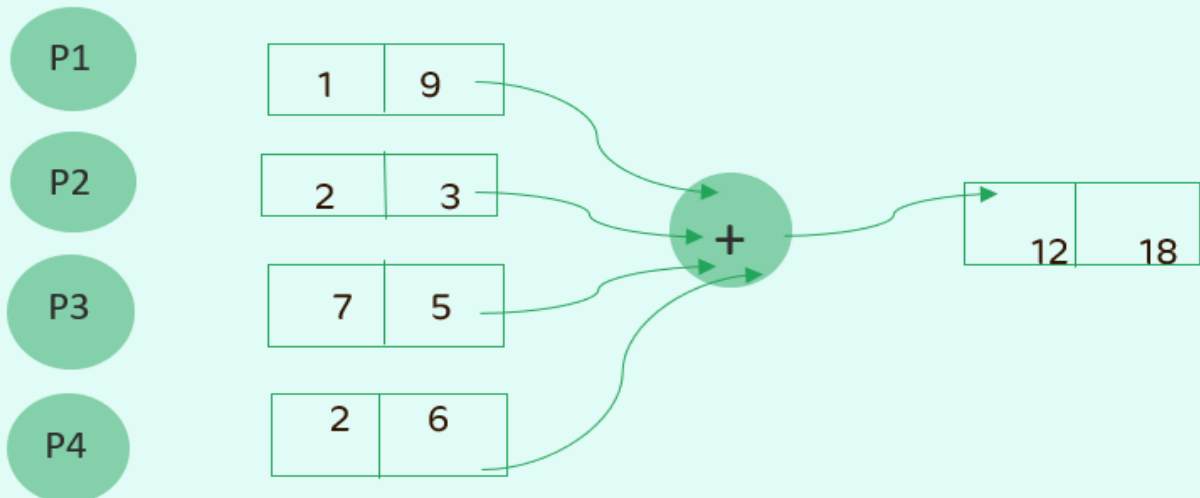


ملاحظة:

لا يمكن القيام ب Scatter بالمرسل و gather بالمستقبل
يتم جمعهم بنفس المسألة عندما يكون عندي مصفوفة أريد توزيعها ومن ثم تجميعها أي
نقوم ب Scatter بالأب ثم بالابناء وأيضاً ب gather بالأب ثم بالابناء

5-Reduce

تجميع أيضاً ولكن مع عملية
أي يمكن القيام بعملية جمع للأجزاء ويكون الخرج مصفوفة بخانتين
أيضا عملية ال reduce يتك وضعها نفسها بالابناء ونفسها بالأب
فيها Parameter يأخذ Id العملية التي أريد القيام بها



إلى اللقاء في المحاضرة القادمة

